

Amortized Analysis

Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

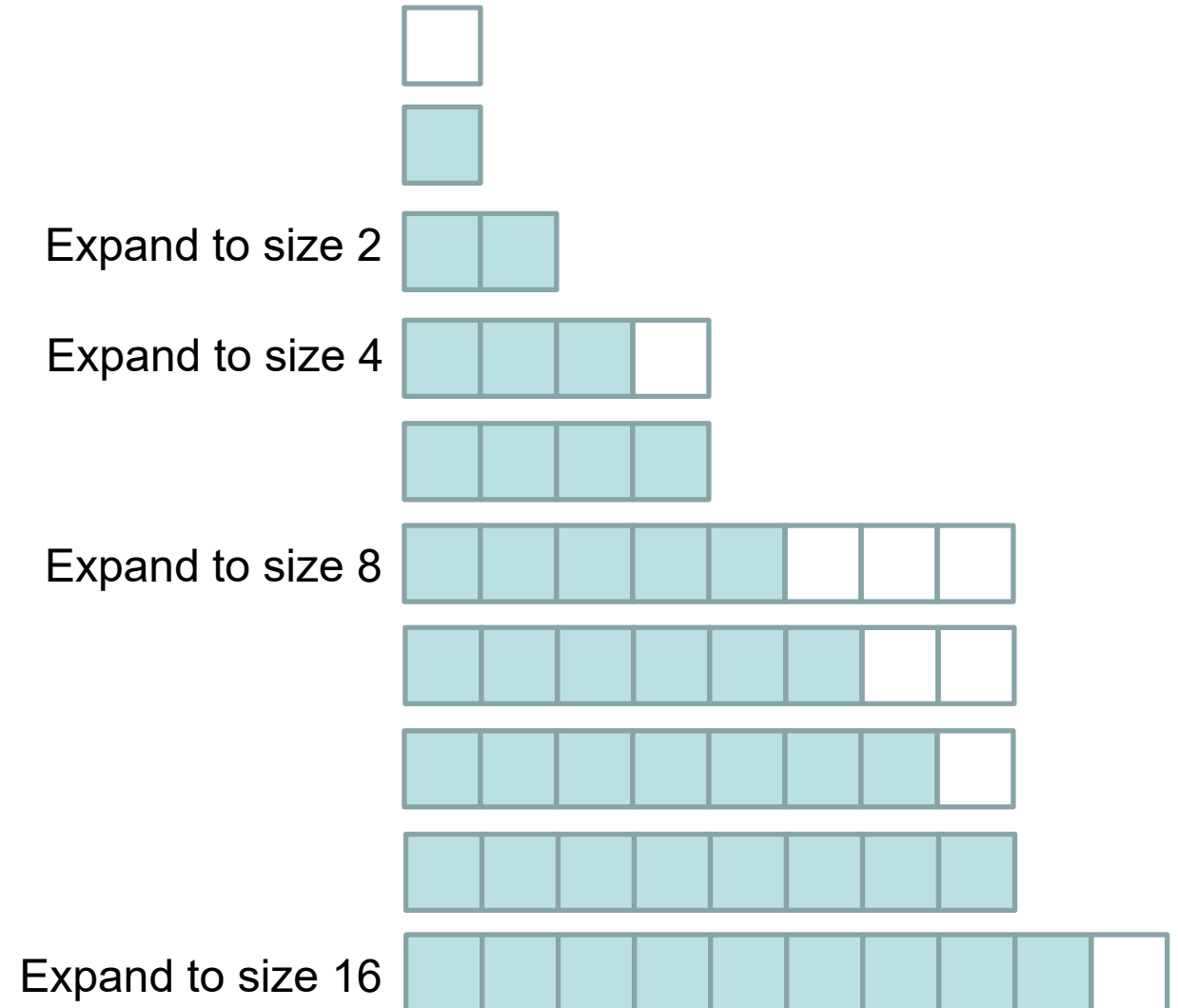


Amortized Performance of Expanding Arrays (Vectors)

- Adding to end of array / vector, multiplicatively expanding when full: $O(1)$ amortized time.

- $O(1)$ amortized means any sequence of k operations takes $O(k)$ time.

(Similarly, $O(\log n)$ amortized time would mean any sequence of k operations takes $O(k \log n)$ time)

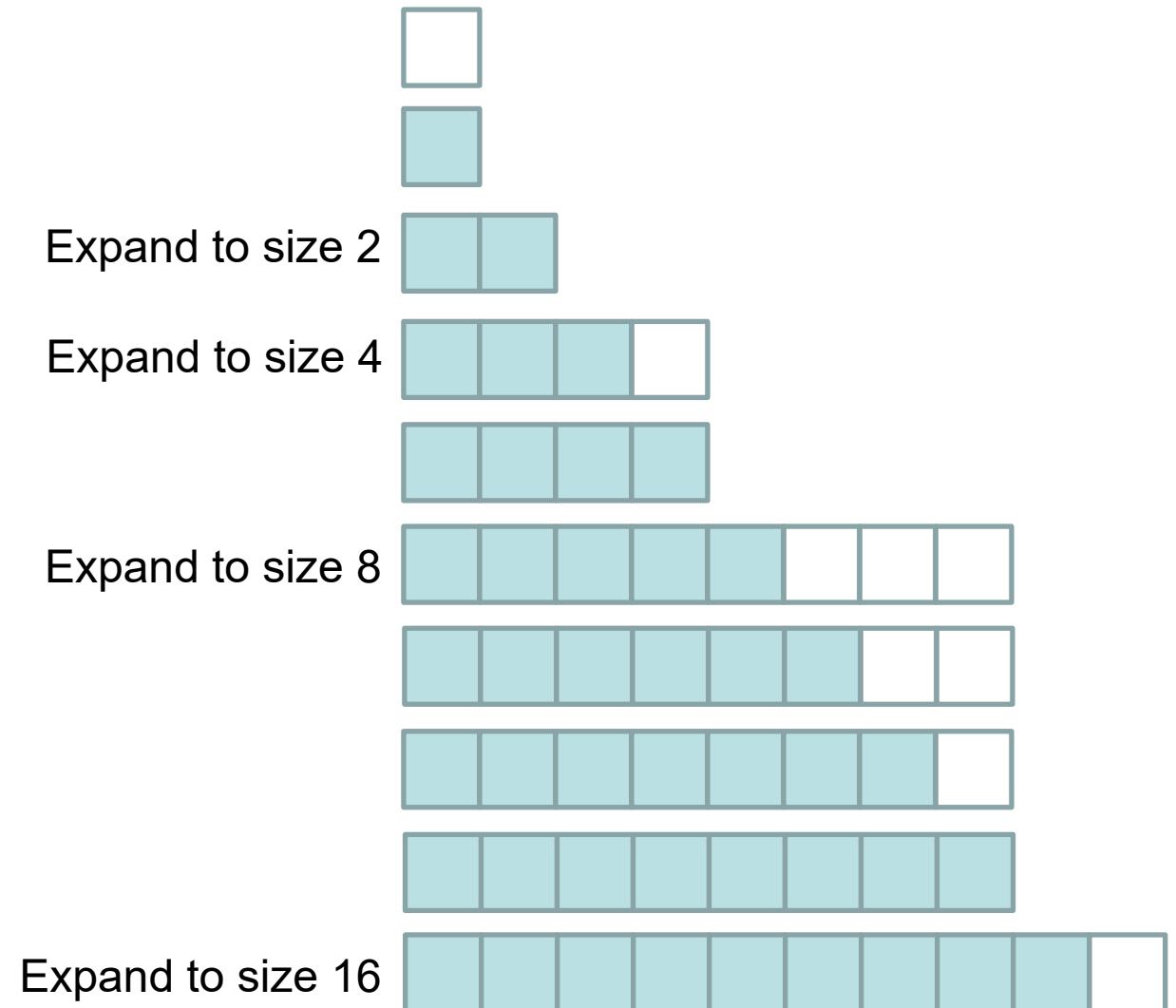


Amortized Analysis : Motivation

- Still a worst-case analysis, just averaged over a sequence of operations.
- Gives a simpler and more accurate way to characterize performance of data structures with highly non-uniform performance profiles (of which there are many!)
 - E.g., “ $\Theta(N)$ worst case vs. $O(1)$ amortized” for a vector.
- Whether operations run in $O(\log n)$ worst case or $O(\log n)$ amortized makes no difference to an algorithm making k repeated calls to the data structure – running time is $O(k \log n)$ in both cases.

Aggregate Analysis: Straightforward, But Limited to Simple Situations

- $O(1)$ amortized means any sequence of k operations takes $O(k)$ time.
- So just show that any sequence of k operations takes $O(k)$ time and we're done! (we did this for the vector in module 1)



Recall: Running Time Analysis from the Perspective of a Data Element

		Elements of Data					
		←-----→					
Steps Taken by Algorithm (in chronological order...)		Elem 1	Elem 2	Elem 3	Elem 4	Elem 5	
	Step 1		1	1			“Swap elements 2 and 3”
	Step 2						
	Step 3				1		“Modify element 4”
	Step 4						
	Step 5	1	1	1	1	1	“Copy elements into a new array”

Time/work done to each element at each major “step” of our algorithm’s execution

In this outlook, we focus less on *when* work happens but rather just *how much* work happens to each element...

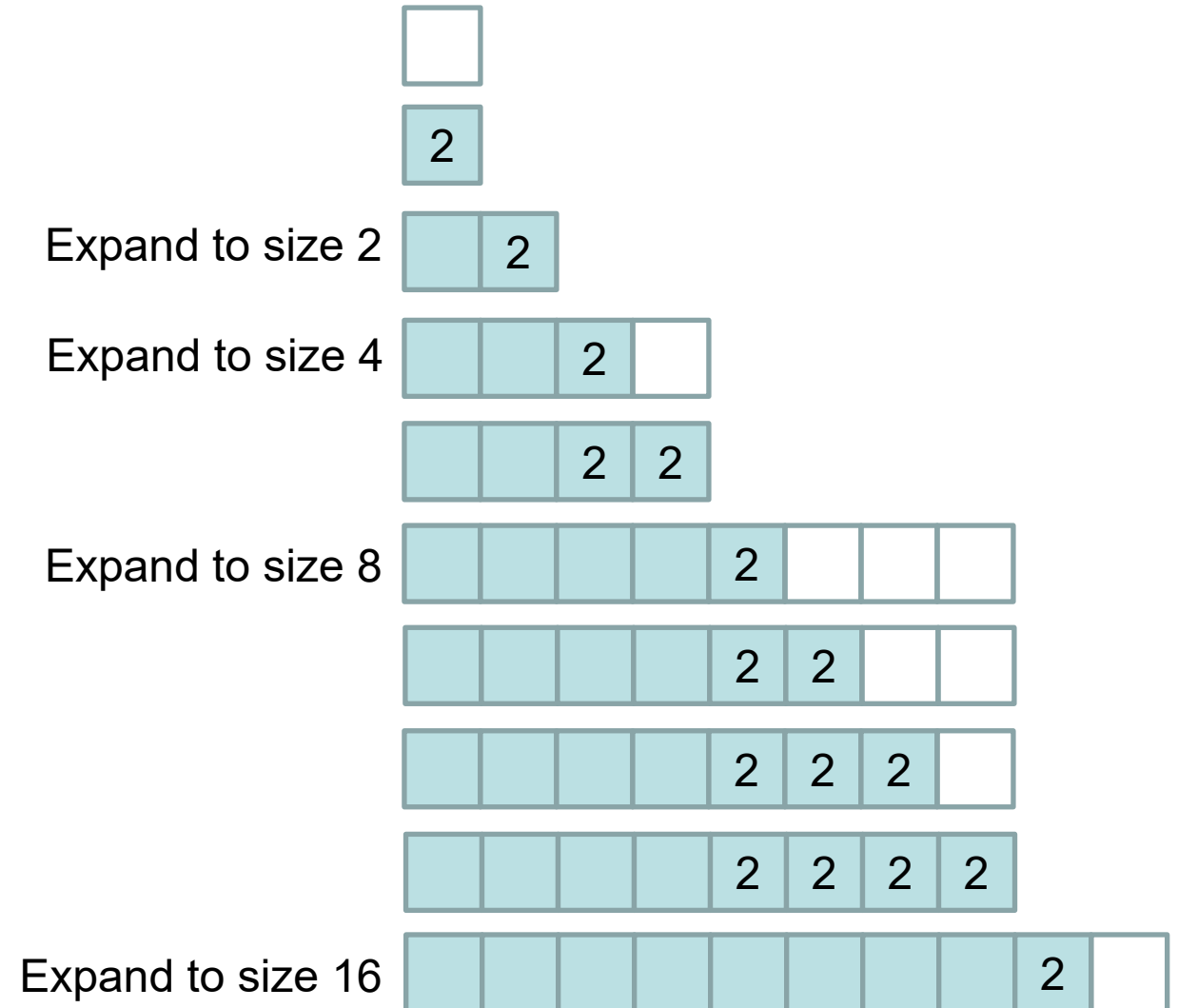
Time Shifting When we Account for Work

- OK to account for work earlier than it happens.
- E.g., think of *insert* costing 2 units (its “amortized cost”), rather than *insert* costing 1 and *delete* later costing 1 (the “actual costs”).
- Sum of amortized costs of a series of operations still gives us an upper bound on their total actual cost.
- Benefit: it doesn’t matter when or even if the delete eventually happens; we’ve still properly charged ourselves for it.
- Example: if data structure periodically does a batch of deletes during an expensive “garbage collection” step, this is “free” in terms of its amortized cost, since we’ve already accounted for it during insert.

Accounting for Work Earlier than It Happens

- Charge ourselves 3 units of work on each insert:
 - 1 unit to pay for the immediate actual cost
 - 2 units as a “credit” to pay off future work involving expansion
- When expansions happen, they are already fully paid for!

	Actual cost	Amortized Cost
Insert	1	3
Expansion	n	0



How Expensive is Your Car to Maintain...?

Jan: \$10

Feb: \$10

...

Nov: \$10

Dec: \$130 = \$10 + yearly \$120 tune-up

Saying “\$130/month in the worst case” doesn’t tell the full story...

Amortization of Costs

Actual Cost

Jan: \$10

Feb: \$10

...

Nov: \$10

Dec: \$130

Amortized Cost

Jan: \$20

Feb: \$20

...

Nov: \$20

Dec: \$20

- So our cost is “\$20/month, amortized”.
- This is a simpler, more accurate description of our cost structure.
- Compare with actually paying \$20/month...

Resizing Memory with “Pushes” and “Pops”

- How about this approach?
 - When the buffer fills up due to too many pushes, double its size.
 - When it becomes less than half full due to too many pops, halve its size (so we don't waste memory)
- NOT good! Every operation can end up taking $\Theta(n)$ time.
- Amortization can't save us when there aren't any intervening cheap operations to amortized over...

“Accounting Method”

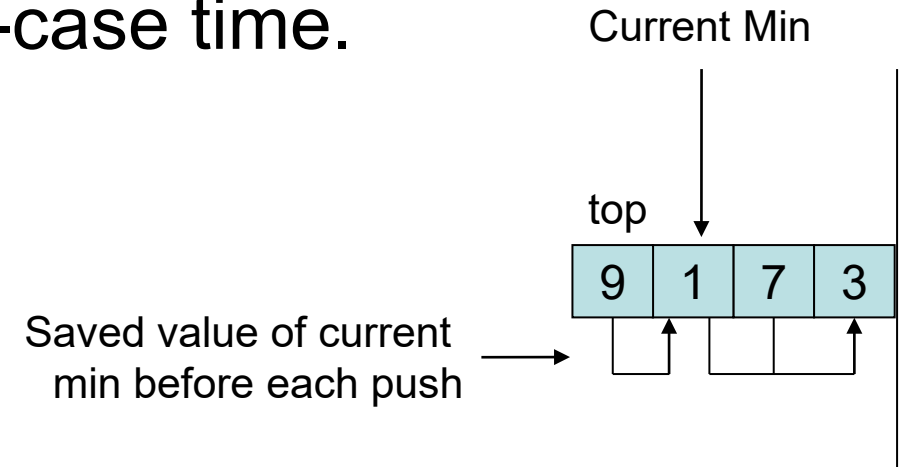
Analysis of Push + Pop

- Charge 3 units (i.e., $O(1)$ amortized time) for each *push* operation.
 - 1 unit for the immediate *push*.
 - “\$2” credit for future memory expansions.
- Charge 2 units per pop (1 unit for the immediate pop, “\$1” credit)



Min-Aware Data Structures

Building a min-stack is straightforward, supporting push, pop, and *find-min* all in $O(1)$ worst-case time.



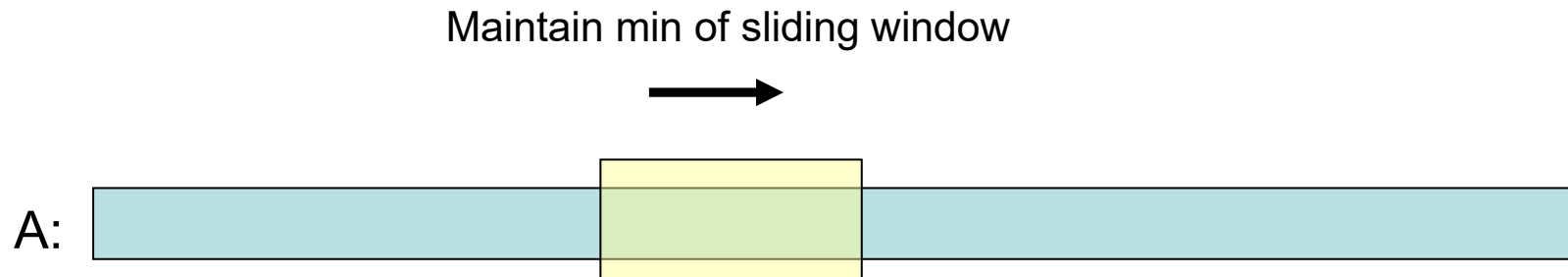
What about a min-queue, supporting insert, delete, and *find-min* all in $O(1)$ worst-case time?

(this is possible, but surprisingly challenging and messy)

$O(1)$ amortized time, however, is much easier to achieve...

Min-Aware Data Structures

Building a min-stack is straightforward, supporting push, pop, and *find-min* all in $O(1)$ worst-case time.

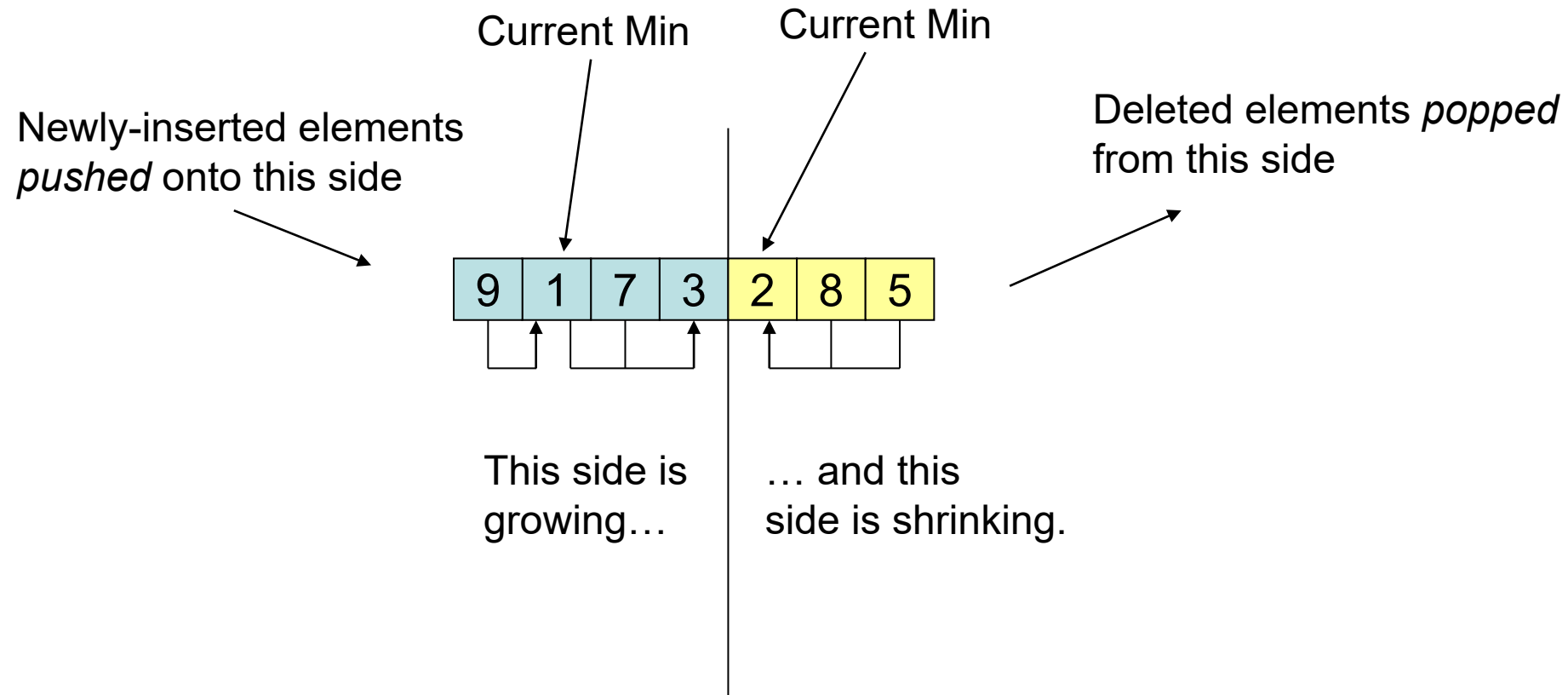


What about a min-queue, supporting insert, delete, and *find-min* all in $O(1)$ worst-case time?

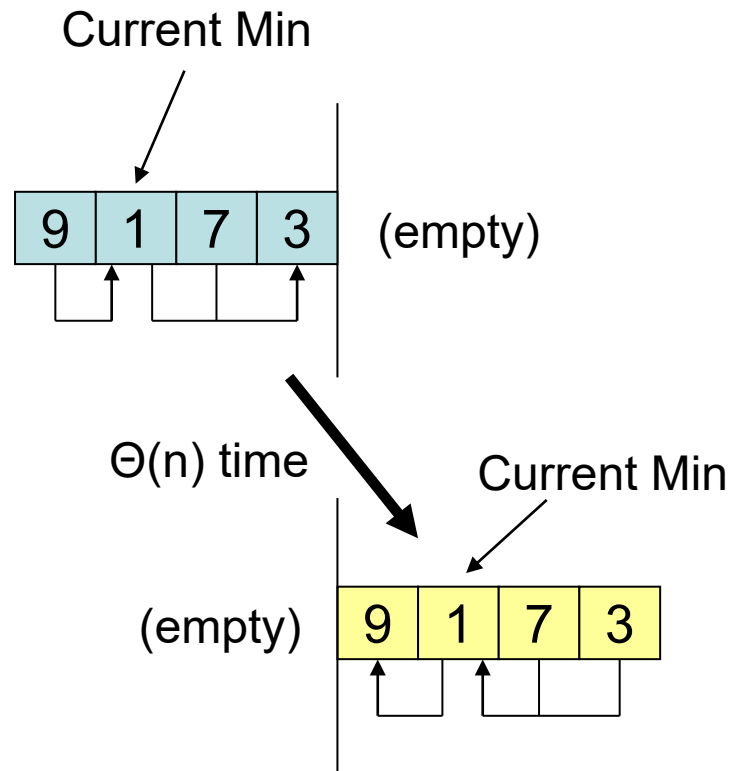
(this is possible, but surprisingly challenging and messy)

$O(1)$ amortized time, however, is much easier to achieve...

The Min-Queue as a Pair of “Back-to-Back” Min-Stacks



Expensive (but Rare) Operations

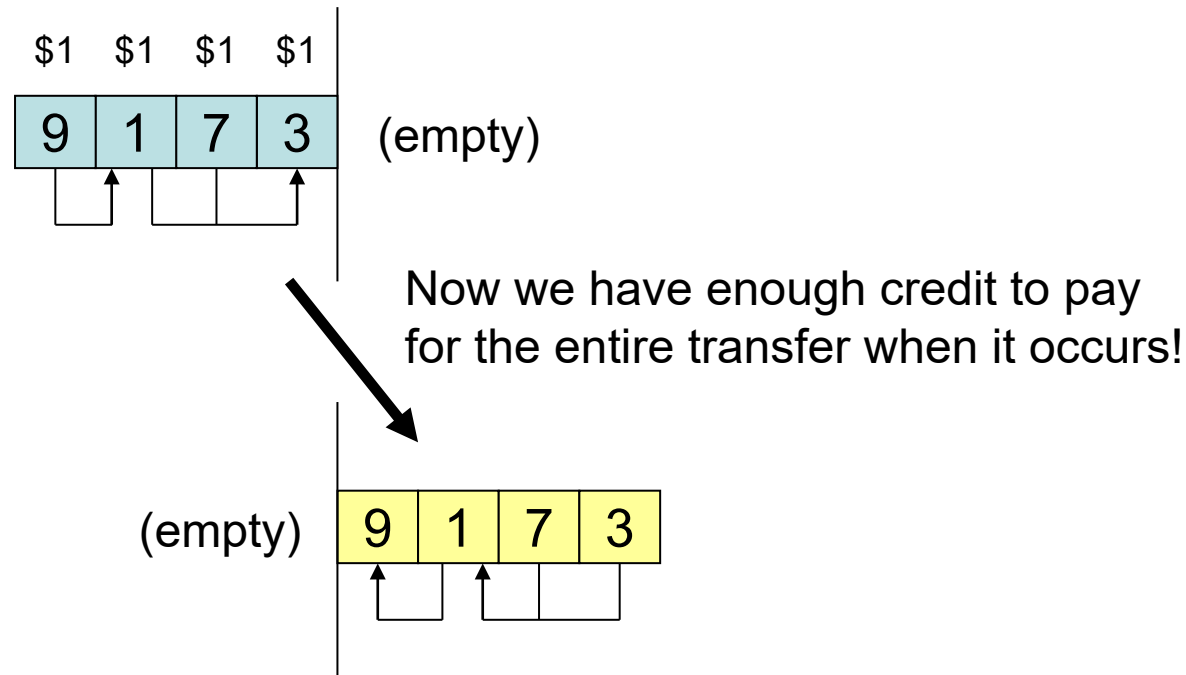


When yellow stack becomes empty, spend $\Theta(n)$ time and transfer blue to yellow.

Worst-case running times: *insert* $O(1)$, *find-min* $O(1)$, *delete*: $\Theta(n)$

Charging Early for Work Done Later

Charge insert 2 units: 1 immediate, with \$1 of “credit” left over.

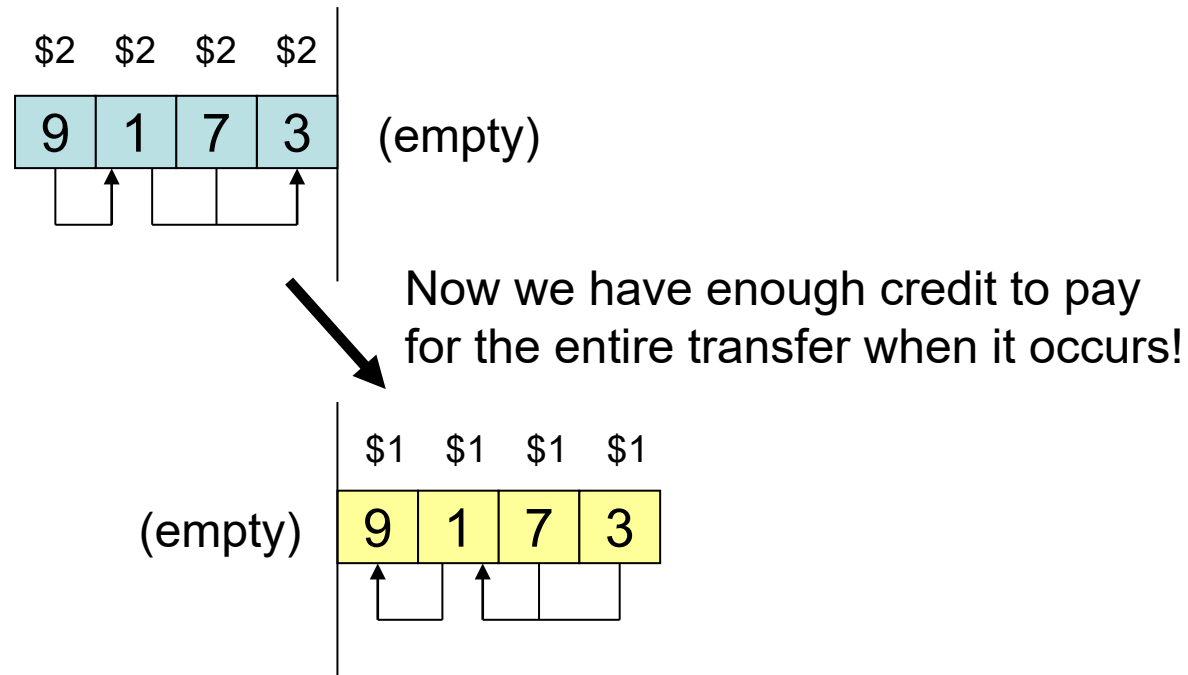


Worst-case running times: insert: 1, find-min: 1, *delete*: 1, **recoloring: n**

Amortized running times: insert: 2, find-min: 1, delete: 1, recoloring: 0

Charging Early for Work Done Later

Charge insert 3 units: 1 immediate, with \$2 of “credit” left over.



Worst-case running times: insert: 1, find-min: 1, *delete*: 1, **recoloring: n**

Amortized running times: insert: 3, find-min: 1, delete: 0, recoloring: 0

Articulating The Same Idea With a “Potential Function”

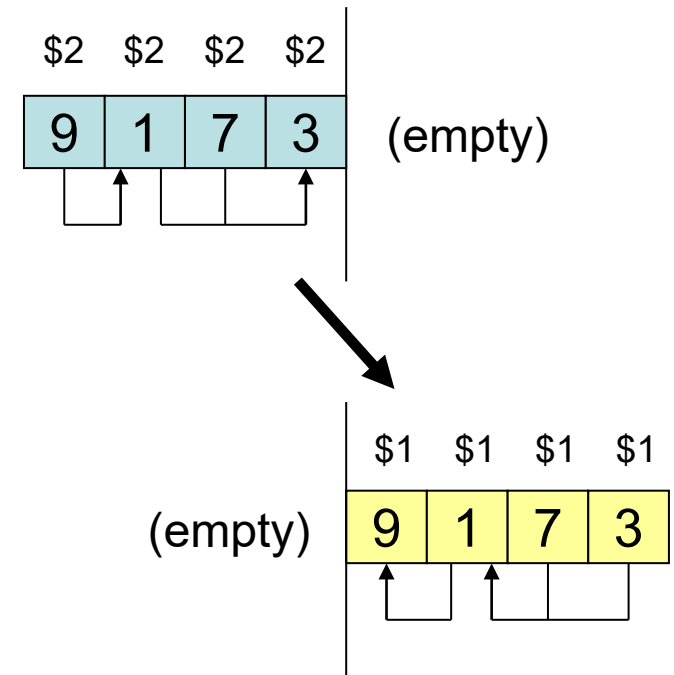
Let $\Phi = 2(\# \text{ of blue elements}) + (\# \text{ yellow elements})$.

Amortized cost of an operation = Actual cost + $\Delta\Phi$

E.g.,

Am cost(insert) = 1 + 2 = 3

Am cost(recoloring) = $n - n = 0$



Worst-case running times: insert: 1, find-min: 1, *delete*: 1, **recoloring: n**

Amortized running times: insert: 3, find-min: 1, delete: 0, recoloring: 0

Articulating The Same Idea With a “Potential Function”

Let $\Phi = 2(\text{\# of blue elements}) + (\text{\# yellow elements})$.

Amortized cost of an operation = Actual cost + $\Delta\Phi$

E.g.,

Am cost(insert) = 1 + 2 = 3

Am cost(recoloring) = $n - n = 0$

For any sequence of operations:

$$\begin{aligned}\sum \text{amortized costs} &= \sum \text{actual costs} + \sum \Delta\Phi \\ &= \sum \text{actual costs} + (\text{leftover potential at end}) \\ &\geq \sum \text{actual costs}\end{aligned}$$

